

LinkedIn & Google Account Creation & Login

Overview & Rationale

To improve user onboarding and reduce friction, we will implement **social login options** using LinkedIn and Google OAuth, alongside the existing email/password signup. Many users (especially our target demographic of young professionals and graduates) prefer using their existing accounts to sign up for services. Offering “Continue with Google” and “Continue with LinkedIn” will allow new users to register for OkBuddy in a **single click** (no need to manually enter email or choose a password), and let returning users log in faster. LinkedIn integration is particularly relevant as our product centers on career development – logging in with LinkedIn can autofill profile information in the future (though for now, we primarily use it for authentication). Google is ubiquitous and covers users who may not have LinkedIn or prefer using their Gmail account. By supporting these, we address a key requirement for a simple registration process and meet user expectations for convenience, while maintaining security through trusted providers.

UI/UX Design for Social Login

We will update the login and registration interfaces to prominently include the option of social sign-ins, without removing the traditional method. Key design points:

- **Combined Login/Signup Interface:** On the login page (which currently has email and password fields and a “Đăng nhập” button as seen in the provided UI screenshot), we will add two **social login buttons**: “Continue with Google” and “Continue with LinkedIn”. These can be placed above the email/password form or below it, with a visual divider or the word “OR” to make it clear the user can choose one or the other. For example:

Prototype layout: “Sign in with Google” and “Sign in with LinkedIn” buttons added above the standard email login form.

The buttons should be styled in accordance with each provider’s branding guidelines:

- The Google button will use the Google logo and either say “Sign in with Google” or “Continue with Google” (depending on common practice, and localized: e.g., in Vietnamese, “Đăng nhập bằng Google”). It typically has a white background with a border and the Google multi-colored logo.

- The LinkedIn button will use the LinkedIn logo (white “in” on blue background) and text “Sign in with LinkedIn” (Vietnamese: “Đăng nhập bằng LinkedIn”) in appropriate font/color. Alternatively, if we use a custom style, at least include the LinkedIn logo icon.

Both buttons should be of equal width and a noticeable size, to stand out as a primary option. We will ensure the style aligns with our design system – likely using our standard button shapes (rounded corners as per our guidelines) , while still incorporating the brand colors/logos of the providers for recognition.

- **Signup Flow:** For the registration page (“Đăng ký”), similarly provide the “Continue with Google/LinkedIn” options. If we have a unified page for login & signup (some apps toggle between them), the social buttons serve both purposes: new users can register via them, existing users can log in. If our flow is separate, the buttons on signup will initiate account creation via social. The text on the buttons might be “Sign up with Google” vs “Sign in”, but often “Continue with Google” works for both cases. We will clarify via a subtext if needed: e.g., “We’ll create an account if you’re new, or log you in if you already have one.” However, that might be too verbose for UI; instead, just using “Continue...” phrasing covers both scenarios seamlessly.
- **One-click Experience:** The aim is that clicking these buttons triggers the OAuth flow. If the user is already logged into Google/LinkedIn in their browser, they might not even see a login prompt (just a consent screen), making the process extremely quick. On return from the OAuth, the user will be logged into OkBuddy immediately (or taken through one small additional step if needed, like choosing a username if we require that beyond email – but since OkBuddy likely uses email as identifier, we might not need extra info). We will show a brief loading indicator or disable the social button after clicking to indicate progress (“Connecting...”) until the redirect happens, to avoid confusion if there’s a slight delay.
- **Placement and Visibility:** We will emphasize these options but also not force them. The email/password fields remain for those who prefer it or don’t have the other accounts. The UI should be balanced: perhaps a subtle text or line separating “Social Login” from “Email Login”. For example:

[Google button]

[LinkedIn button]

_____ hoặc _____ (or)

Email: [_____]

Mật khẩu: [_____]

[Đăng nhập] [Quên mật khẩu?]

Chưa có tài khoản? Đăng ký ngay

- This bilingual mix shows using “hoặc” (or) for Vietnamese locale. In English UI, it would show “_____ or _____”. This clearly delineates the two methods.
- **Mobile Considerations:** If our web app is mobile-responsive, ensure the buttons are tap-friendly (large enough). Also, consider the case of an embedded webview (though not sure if we have mobile app or just web) – OAuth might not work well in an embedded webview without opening a browser; but that’s likely fine for now (scope is web). The design should degrade gracefully if the user’s device can’t handle the redirect (maybe provide a direct link).
- **Localizable Text:** All new text on these buttons or error messages will be added to our resource files for translation (e.g., English and Vietnamese) . “Sign in with Google” vs “Đăng nhập bằng Google” etc., as noted. Provider names like “Google” or “LinkedIn” we will likely keep in English as brand names, but the verb and phrasing around it will be translated.

Google OAuth Integration

Implementing Google login involves using OAuth 2.0 via Google’s APIs:

- **OAuth Credentials:** We will register OkBuddy with Google API Console to obtain a Client ID and Client Secret for OAuth. The allowed redirect URI will be our website’s callback endpoint (e.g., <https://okbuddy.com/auth/google/callback>). This ensures Google will redirect back to our site after authentication.
- **User Clicks “Continue with Google”:** On click, our frontend will initiate the OAuth flow. This can be done by redirecting the browser to Google’s authorization URL with appropriate query parameters (client_id, redirect_uri, scope, state, etc.). Scope wise, we need the user’s email and basic profile info. The minimal scopes would be openid email profile. This gives us an ID token with email, and possibly name. We aren’t requesting any sensitive scopes beyond that (no Google Drive or anything). The user will see a Google screen asking to choose an account and grant OkBuddy access to their basic profile info.
- **Google Redirect & Backend Handling:** Google will redirect back to our specified callback with an authorization code (if the user consented successfully). Our backend server (or dedicated auth service) will handle this callback. It will:
 - Receive the code (and verify the state for CSRF protection).
 - Make a server-side POST request to Google’s token endpoint to exchange the code for an access token (and an ID token). We include the client secret in this

step, so it must be done securely on backend.

- Google responds with an ID token (JWT) and possibly a separate access token. The ID token contains the user's Google Account info (subject, email, email_verified, name, picture, etc.). We will **verify the ID token** (check signature and that it's intended for our client_id) as an extra security measure.
- Extract the user's email from the token (this should be a verified email from Google). Also get their full name (given_name, family_name) if available, and profile picture URL if we want to use it (not critical for auth, but could save as profile image).
- **Account Lookup/Creation:** With the Google-provided email, our backend will check if an OkBuddy account exists with that email:
 - *Existing Account:* If found, we proceed to log the user in to that account. This covers both cases where the account was originally created by email/password or via a different social method:
 - If it was a normal account (email sign-up), by logging in with Google we are essentially doing a password-less login to the same account. This is okay because ownership of the Google email implies ownership of that account's email. We will link the Google OAuth info to this account for future (so next time, they can use Google again to login). Linking means storing the Google account's unique ID (sub) in our database tied to the user record, and noting that Google is authorized for this user. We should also mark the email as verified if it wasn't already, since Google verified it.
 - If it was an account via LinkedIn with the same email, similarly, we now link Google login to it. The email is our primary key, so in effect it's the same user. From the user's perspective, they just logged in successfully. In the background, we update the account to record that this user can login via Google as well.

After login, the user is redirected to the post-login page (e.g., dashboard or last visited page). There's no interruption or extra steps because the account already existed.
 - *New Account:* If no account with that email exists, we will **create a new account** in our database:
 - Generate a new user record with that email. We'll set the user's name in our database to the full name from Google profile (to personalize their experience – e.g., greeting them by name). If the Google response has a

profile picture URL, we might store it or download it as their avatar (optional, can be done later or prompted).

- Since this user authenticated via Google, we **do not need a password** for them. The account can be marked as “Google-auth only” (though we can allow password to be set later). We should consider the email verified by default (since Google only gives it if verified).
 - We assign any default settings or free trial status as we would for normal sign-up. We might send a welcome email, but note that sending a welcome email with login info might not include a password since they didn’t set one; instead it can say “You signed up via Google – just use Google to log in next time!” or offer a link to set a password if they want.
 - Immediately create a session for the user (log them in) so that after the redirect they are in their new account. From the user’s view, it was a one-click signup. Optionally, if there’s additional onboarding (like asking them to fill some profile details), we direct them there now.
- **Conflict Scenarios:** If an account *did* exist but under a different method, we handle it as described (no duplicate creation). There is a subtle case: what if an account existed with the same email but was unverified or was a placeholder? Generally, we treat it as the same account. If the account was disabled or deleted previously, logging in via Google could resurrect it – we may need a policy: e.g., if account was soft-deleted and email matches, perhaps we undelete and let them in (to avoid confusion “account exists but you can’t use it”). If an account was banned, we should prevent login even via social (show appropriate message). These are edge admin cases.
 - **Security:** We rely on Google for initial authentication, which is very secure (Google handles password, 2FA, etc.). We must secure our callback (check state param to prevent CSRF, verify tokens). We’ll use HTTPS for all these flows. We will not store the Google access token long-term (unless we plan to fetch more data later, which we currently don’t). We mainly use the ID token for identity. Possibly, we store the Google refresh token if we ever need to use Google APIs on user’s behalf (not in scope now). For auth, just storing Google’s unique user ID and token expiration is enough.
 - **User Experience Post-Login:** After a successful Google login or signup, the user goes to the same place as if they logged in normally. If it’s the first time, maybe trigger a welcome tutorial or at least a message: “Welcome, [Name]! Your account has been created via Google.” There’s typically no need for them to verify email since that’s done. If we do send a welcome email, it’s purely informational (“Thanks for joining OkBuddy”). We should also consider that some users might have signed up with Google accidentally when they intended to use email/password. It might be helpful to provide an option in account settings: e.g., “Add a password to my account” if they want to also log in via

email in the future (for instance, if someday they lose access to Google). This is more of an account management feature – at minimum, they can always do a password reset using the same email to set a password.

LinkedIn OAuth Integration

LinkedIn login will be implemented in a similar fashion, with a few LinkedIn-specific considerations:

- **OAuth Setup:** We need to register OkBuddy as a LinkedIn app through the LinkedIn Developer portal to get a Client ID/Secret. We must define redirect URI(s) in their config as well. LinkedIn requires specifying the scopes and getting approval for certain data access. For basic login, the scopes needed are:
 - `r_liteprofile` – to get basic profile info (name, profile ID)
 - `r_emailaddress` – to get the user's email address

LinkedIn's consent screen will show these requests. We should ensure our app description and name clearly indicate OkBuddy so users trust the request.
- **User Clicks "Continue with LinkedIn":** Frontend will redirect the user to LinkedIn's authorization URL (with our client id, scopes, state, etc.). The user may need to log into LinkedIn if not already, then will see a permission consent (LinkedIn will say "OkBuddy is requesting your profile info and email" in some form).
- **LinkedIn Redirect & Backend:** After consent, LinkedIn redirects to our callback with a code (similar to Google's flow):
 - Our server receives the code (verifies state for security).
 - Server makes a POST request to LinkedIn's token endpoint to exchange the code for an access token. LinkedIn returns an access token (often no ID token; LinkedIn doesn't use OpenID Connect in the same way, so we have to use the token to fetch user info).
 - Using the access token, we call LinkedIn's API endpoints:
 - GET `https://api.linkedin.com/v2/me` – to get profile data (this gives us user's LinkedIn ID and name). Specifically, fields like `firstName`, `lastName` (which are localized name objects) and an `id`.

- GET

[https://api.linkedin.com/v2/emailAddress?q=members&projection=\(elements*\(handle~\)\)](https://api.linkedin.com/v2/emailAddress?q=members&projection=(elements*(handle~))) – to get the primary email. LinkedIn returns the email in a nested structure (handle~ contains the actual address).

We parse these responses to extract: user's email and full name. Email is crucial; name is good to have. The LinkedIn profile may not directly give a profile picture URL without additional scope, but that's fine for now.

- Now we have the email (assuming the user granted email permission, which they should have if they accepted – if not, we have a problem scenario covered below).

- **Account Lookup/Creation:** This proceeds just like in Google's case:

- *Existing Account with that Email:* Log the user in to that account.
 - If it was a password account, we link the LinkedIn login as an alternate method. The next time they could use either method. We store the LinkedIn ID in their user record (to avoid creating duplicates in future). We consider their email verified now (LinkedIn provided it).
 - If it was a Google-created account (same email), we link LinkedIn as well. Now the account has two social logins attached.
 - In all cases, session is created and the user is logged in seamlessly.
- *No Account Exists:* Create a new account using that email:
 - Fill in name from LinkedIn (LinkedIn gives first and last name; we'll concatenate for full name, or store separately if our DB has fields).
 - Mark as verified email, no password needed (similar to Google).
 - Possibly store their LinkedIn ID for reference. We likely won't have a username for them unless we use email as username.
 - Immediately log them in (session).
 - Send welcome email if applicable. Also, since LinkedIn is career-focused, we might consider asking if they want to import some profile data (future feature: fill their work experience from LinkedIn automatically). But that is beyond the scope here – just note it as a possibility in product roadmap.

For now, we only use LinkedIn for auth, not data import.

- **Conflict Handling:** The same email conflict scenarios apply as described in Google section. If email is already in use, we do not create a new account. This prevents duplicates and confusion. We should ensure to handle the scenario where:
 - A user *accidentally* uses different emails on Google vs LinkedIn (e.g., Gmail for Google, a work email for LinkedIn) and ends up with two accounts. That's not something we can solve automatically (since emails differ, we treat as separate accounts). The user might not realize at first. If they contact support, we could offer to merge accounts, but automated merging without email match is not done for security reasons. We can mitigate by encouraging users to stick to one method or explicitly allow linking in settings where they log in and add another login method.
- **Error Handling for LinkedIn OAuth:**
 - If the LinkedIn flow fails (user denies permission or something goes wrong), we redirect them back to our login page with an error message: e.g., "LinkedIn login was not completed." We might log the reason if given (LinkedIn might return an error and error_description in callback if user denied). The UI message should be simple: possibly the same generic "Social login failed or was canceled. You can try again or use your email to sign in."
 - If we get a token but the subsequent API call to fetch email fails (it shouldn't if scopes are correct), treat it as failure – without an email we cannot proceed. In such rare case, we should inform the user: "We couldn't retrieve your email from LinkedIn. Please sign up manually or try a different method." This scenario might happen if the user's LinkedIn account has no confirmed email (unlikely) or if our app wasn't granted the email scope.
 - LinkedIn's API can be slower; we should ensure a reasonable timeout and user feedback. The user might sit on our callback loading state for a second or two while we fetch data. Possibly show a spinner "Signing you in..." during that. If it times out, show an error and allow retry.
- **Security:** We will protect the LinkedIn integration similarly: verify state param, use HTTPS, and handle tokens carefully. We should also consider that LinkedIn tokens expire (usually quickly, within hours). Since we're not using the token beyond initial login, that's fine. If we want to keep the user "logged in" to OkBuddy, that's via our own session mechanism (cookie or JWT for our site) – independent of LinkedIn's token. The LinkedIn ID we store is a long-term identifier for linking; even if token expires, we have the ID to avoid duplicates next time. We must keep the client secret safe and not expose it on

frontend.

- **Post-Login Experience:** Similar to Google, the user lands in the app, possibly greeted by name. If it's a new account via LinkedIn, one idea: we could show a prompt like "Import your LinkedIn profile to build your CV faster!" as a next step (though building that import feature is future work). At minimum, they've saved time on registration. We might want to let them know they can also log in with Google if they want to link it (but that's optional – usually account linking is done in an account settings page, which we might implement later).

Account Linking & Conflict Resolution Logic

It's crucial to detail how we handle accounts when the same user might use different login methods. Our guiding principle is **one user account per email address** on OkBuddy. Email is the unique key that ties everything together.

- **Unique Email Constraint:** Our system will enforce that each email can only be associated with one OkBuddy account. This means during any account creation flow (manual or social), we check for existing email. If found, we don't create a new account – instead, we attach the login method to the existing account.
- **Login via Social when Email Already Registered:**
 - *Scenario A:* User has a normal account (email & password) under user@example.com. Later, on the login page, they choose "Continue with Google" and select the Google account user@example.com. Our backend sees that email exists. In this case:
 - We **do not create** a new account. We log the user into their existing account. The system can treat this as a login attempt to an existing account, just with an alternative credential.
 - We should record that this account is now linked to Google. Concretely, in the user's record, a field like google_id or an entry in a "login_providers" table is added. This way, next time they use Google login, we know immediately which account to fetch (we can also lookup by email again; both ways work).
 - If the existing account had no password set (e.g., maybe an admin created an account or something), this gives them a way in. If it did have a password, both methods now work.

- From the user's perspective, after the Google OAuth, they're simply logged in – they might not even realize that an account already existed; they likely just think “great, it logged me in”. We could show a subtle message like “Linked to existing account” but it may not be necessary unless we foresee confusion.
- *Scenario B*: User initially signed up with Google (account email is user@example.com, no password). Later they try to log in manually by entering that email and clicking “Forgot password” (since they don't have one). This is allowed: the password reset will send an email to user@example.com. If they set a password, now their account has both Google and a password login. This is fine and by design – some users like to have a password as backup.

Alternatively, if they tried to sign up anew with email+password using user@example.com, our system on form submission should catch “email already in use” and not allow it. It should prompt them that this email is already registered. Perhaps it can hint if it's linked to Google: “This email is already registered. You may have signed up via Google – try logging in with Google, or use Forgot Password to set up a password login.” This message will help users who forgot they used Google.

- *Scenario C*: User signed up with LinkedIn (email user@something.com). Later tries Google with same email. Outcome is the same: they'll log into that account. Or vice versa (Google first, then LinkedIn). Essentially, as long as the email is consistent, they end up in the one account.
- *Edge*: What if a user has two social accounts with different emails (e.g., personal Gmail vs work email on LinkedIn) and they use both? They will inadvertently create two OkBuddy accounts. If the user realizes and wants to merge them, currently we have no automatic detection. They would need to change one of the account's email to match or contact support. This spec will not implement merging, but we will design to minimize confusion. For example, our UI can show the logged-in user's email somewhere; if they notice it's different, they might realize and choose one account to stick with. In future, we could allow adding a secondary email or linking accounts with different emails via a verification process, but that's outside current scope.

- **Automatic Account Creation vs. Linking:**

- On **social login** attempt, we are effectively doing “login or sign-up”. We will always attempt to find an account by email first. Only if none is found do we create a new one. This covers the requirement “*social login automatically creates an account if no existing email match is found*” – yes, we create on the fly.

- On the flip side, if a user is on our **manual signup form** and enters an email that already exists via a social account, as noted, we should block creation with a clear error. This prevents duplicate accounts. The error message can guide them: if the email domain is gmail.com and our records show it's a Google-linked account, the error might say "This email is linked to a Google login. Please use 'Continue with Google' to log in." If it's a LinkedIn-linked email, say similar. However, to avoid revealing too much info (for security, some sites avoid confirming an email is registered to prevent user enumeration), we might use a generic message like "Email already registered. Try logging in or use a different email." Since the user themselves is providing the email, it's not too risky to hint the method, but we should be cautious not to reveal to a malicious actor that an email has a Google account. Perhaps simpler: "Email already in use. If you have an account, please log in or reset password." That covers all cases without specifying the provider. The user will figure out if they recall using Google.
- **Session Management:** Once logged in via any method, the session works the same. Logging out will sign the user out of OkBuddy (destroy session). It will not log them out of Google or LinkedIn – that's fine/expected. If they click the social login again next time and are still logged in to that provider, they may not see a login prompt. If they had logged out from OkBuddy but not Google, hitting "Continue with Google" might instantly log them in without asking credentials (because Google remembers them). This quick login is a nice benefit, but users should be aware if they share a computer – we might mention "If you're using a public computer, be sure to log out of your Google/LinkedIn as well." But that can be in a help section, not necessarily a prompt each time.
- **Multiple Providers Linked:** Our account model should allow that a single account can have multiple OAuth logins associated (e.g., a user could potentially link both Google and LinkedIn, plus a password). This is good for flexibility. The primary key is email, but we will store provider IDs too. Implementation-wise, we might have a table like user_auth with columns: user_id, provider (google/linkedin), provider_user_id (Google's sub or LinkedIn's id). On login, we can either find by email or by provider_user_id if we store that. However, since we always get email and we ensure email is unique, looking up by email is straightforward. Storing the provider_user_id is mainly to prevent creating a second account if somehow the same email is used by two different Google accounts (which can't happen – Google's one email per account) or just for record. It's also useful if in future we allow linking while having different emails (less likely).
- **Handling Revocation:** Not exactly a conflict, but what if a user later revokes OkBuddy's access in their Google/LinkedIn settings? If they do so, our stored link might become invalid (the next login attempt via that provider will require re-consent). We should handle that gracefully – basically, they'd just be treated as a new OAuth login attempt and ask again. This doesn't affect the account existence; the email is still there. Worst case, the user would need to log in with password and re-link, or just do the OAuth again which prompts them. This is a rare edge case and doesn't need special UI, just

awareness.

- **Email Change Implications:** Currently, our account identity is email. If we allow users to change their email in settings in the future, we'd have to also update the link or at least warn them that social logins might break (since Google will still send the old email). One way some systems handle this is not by email at all, but by storing the OAuth provider ID as main link. But since we might have multiple methods, email is a common denominator. We likely won't implement email changing at this stage, but if we do, we should check: if a user with Google login changes their email in OkBuddy, that doesn't change their Google account email. Next time they try Google login (which sends old email), it won't find because OkBuddy now has a different email. So, better to restrict or handle that carefully (maybe require re-linking social accounts if email changed). This is advanced account management beyond initial scope, but documenting it is good. For now, we assume email change is either not allowed or rare (maybe via support) and can be handled manually.

Edge Cases & Security Considerations

Finally, enumerate specific edge cases and how to handle them to ensure a robust social login feature:

- **User denies consent:** If a user clicks "Continue with Google/LinkedIn" but then denies permission at the OAuth screen or simply closes the window, they'll be returned to our app's callback with an error (or not returned at all if they closed it). We must handle this gracefully:
 - If the callback URL has an error parameter (e.g., `error=access_denied` for LinkedIn or similar for Google), we interpret that as "user canceled". In that case, we redirect the user to the login page (or wherever they started) and show a **non-intrusive notification**: e.g., a small red banner or toast: "Google login was canceled." The login page remains, so they can try again or use another method. No account changes happen in this case (since we didn't get any data).
 - If the user closed the window and never hit our redirect, our app might be waiting. Usually we open OAuth in the same tab or a popup. If it was same tab, they won't come back without completing; if it was a popup, our main page might poll or just wait. To keep it simple, we likely do redirect in the same tab for web. So no additional handling needed beyond the callback.
- **Missing Email:** As mentioned, if Google or LinkedIn doesn't provide an email, we have an issue. With Google, using the correct scope ensures we get the email (and we ask for verified emails only). With LinkedIn, we requested `r_emailaddress`; if the user did not grant it (they can't selectively deny one scope and accept another – it's all or nothing on

LinkedIn's consent), so if they consent we have it. If we ever fail to get an email (e.g., LinkedIn API call fails due to some API issue or the user's account has no primary email which is extremely unlikely), we cannot proceed to create account. We would show an error to user as noted: something like "We couldn't retrieve your email from LinkedIn. Please sign up with email or try Google." and perhaps log this incident. We should also not leave them in a limbo – ensure they can still sign up manually if that method fails.

- **Provider-specific Errors:**

- Google could return errors like `invalid_client` (if our setup is wrong) – that's on us to test properly. If it happens in production, we'd need to fix config; user can't do anything, so just generic error.
- LinkedIn might have rate limits (they allow X logins per day or something unusual). We don't expect hitting those with normal usage.
- If either provider's service is down (rare), social login will not work at that time. In that case, users still have the email login as fallback. If a user only ever used social login and that provider is down, they might be stuck – which is why giving them the ability to have a password (via set password or having done forgot password) is useful. But as a mitigation, if Google is down, likely many sites are affected; user can try later. We could detect and put up a notice if it's a known outage (not necessary now).

- **CAPTCHA / Anti-bot:** Since social logins bypass our normal registration form, we might lose an opportunity to captcha-verify a new user. However, Google and LinkedIn logins are themselves less likely to be used by bots (harder to automate fully because require real accounts). It's possible an attacker could create many Google accounts and script login to create many OkBuddy accounts. To mitigate mass fake signups, we can implement checks: e.g., limit the number of accounts created per IP per hour, or use Google's risk assessment API on the callback. But given the complexity, we might rely on general monitoring for unusual spikes. We did have a requirement for CAPTCHA on registration – perhaps we skip CAPTCHA if using social login since the provider already adds friction. If needed, we could integrate something like Google's reCAPTCHA in the OAuth flow, but that's likely overkill. We consider social logins as somewhat *verified* since they come from established accounts (especially LinkedIn – usually professional accounts, less likely bots).

- **Email Verification:** In the normal flow, after email sign-up, we might send a verification email to confirm ownership. With social logins, we inherently trust the provider's verification (Google only provides email if it's verified; LinkedIn emails should also be actual). Therefore, we will mark social signup accounts as verified to avoid sending a redundant email confirmation. We will, however, send a welcome email or an account link email as mentioned. If our policy is to always verify new emails, we can consider the

OAuth an automatic verification. We should make sure not to accidentally send a “Please verify your email” to a Google sign-up user, as that’s unnecessary friction.

- **Login UI States:** Ensure our UI properly handles if a user tries to log in with a wrong password for email, then switches to social – the error for wrong password should clear, etc. These are minor UX states to test. Also, after a social login attempt fails and we show an error, the user might decide to use email login; the error message shouldn’t persist wrongly.
- **Logging and Analytics:** We should track how many users use Google vs LinkedIn vs email to sign up and log in. This helps business understand user preference. We can tag accounts with a “signup_method” (google, linkedin, email) for analysis. Also, log login events (provider used, success/fail). Be mindful not to log sensitive info (no tokens, just high-level events).
- **Terms of Service & Privacy:** Since we’re integrating third-party auth, update the Privacy Policy / ToS if needed to mention we collect basic profile info from Google/LinkedIn when users connect those accounts. Also, ensure compliance with Google’s and LinkedIn’s branding guidelines (likely including phrases like “Google” with capital G, not saying we are partnered, etc., and a note like “Sign in with Google is a service provided by Google, Inc.” if required by Google’s TOS).
- **Future Social Integrations:** The structure for Google and LinkedIn should be built in a generic way (perhaps using a common OAuth callback handler that knows which provider is calling back via a state param or separate endpoint URIs). This will make it easier to add more providers later (e.g., **GitHub** for tech users, **Facebook** for broad reach, or **Apple** for iOS users). While not immediately planned, setting the foundation now (like a unified userAuth table as discussed) will ease future expansion.